Studio Nx

백엔드 279 — 패턴 서비스

2026-10-19

백엔드 279 — 패턴 서비스 (services/pattern_service.py)

항목	내용
파일	`studionx/backend/app/services/pattern_service.py`
작업	S1-10a 클립·연결·패턴 API: 패턴 만들기, 노트 저장, 패턴 삭제 서비스 (PRD 02 FR-2196, FR-231)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 278 연결 서비스, 다음 280 클립 · 연결 라우터

1. 하는 일

피아노롤 패턴을 만들고, 노트를 저장하고, 쓰이지 않는 패턴을 지우는 일을 담당합니다. 노트는 패턴 하나에 모두 붙어 있어서, 저장할 때마다 그 패턴의 노트를 보낸 목록으로 통째로 바꿉니다.

```
create_pattern(db, user, data)            멤버십 확인 → 권한 확인 → 빈 패턴 삽입 → 노트는 빈 목록으로 201
replace_notes(db, user, pattern_id, data) 패턴과 멤버 확인 → 권한 확인 → 기존 노트 전부 삭제 → 보낸 노트 삽입
delete_pattern(db, user, pattern_id)      패턴과 멤버 확인 → 권한 확인 → 클립 사용 검사 → 쓰지 않는 패턴만 삭제
```

노트 저장은 `PUT /patterns/{id}/notes` 로 받으며, 본문은 노트 목록을 담습니다. 노트 한 건은 `pitch`(0 이상 127 이하), `start_tick`(0 이상), `length_tick`(1 이상), `velocity`(1 이상 127 이하)를 기본으로 하며, `channel`, `release_velocity`, `is_muted`, `probability`, `velocity_deviation` 은 보내지 않으면 기본값이 채워집니다. 보내는 노트는 최대 5000개이고, 빈 목록을 보내면 패턴의 노트가 모두 지워집니다.

2. 코드

```
def _find(db: Session, user: User, pattern_id: int) -> tuple[Pattern, Project, ProjectMember]:
    """패턴과 그 프로젝트·내 멤버 행을 함께 찾는다. 없거나 남의 것이면 404."""
    pattern = db.get(Pattern, pattern_id)
    if pattern is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Pattern not found.")
    project, member = project_service.get_membership(db, pattern.project_id, user)
    return pattern, project, member
```

`create_pattern` 은 새 빈 패턴을 만들고, `replace_notes` 가 노트를 통째로 교체합니다.

```
def create_pattern(db: Session, user: User, data: PatternCreateRequest) -> PatternResponse:
    """새 빈 패턴 만들기 (FR-2196). OWNER·EDITOR 만. 노트는 나중에 피아노롤로 채운다."""
    project, member = project_service.get_membership(db, data.project_id, user)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    pattern = Pattern(
        project_id=project.project_id,
        name=data.name,
        color=data.color,
        length_tick=data.length_tick,
        swing=data.swing,
    )
```

```
    db.add(pattern)
    db.commit()
    db.refresh(pattern)
    return to_response(pattern, [])


def replace_notes(
    db: Session, user: User, pattern_id: int, data: NotesSaveRequest
) -> PatternResponse:
    """패턴의 노트를 보낸 목록으로 통째 교체한다 (FR-231). OWNER·EDITOR 만.

    (구현: AI — 2026-10-17, S1-08) 지금 붙은 노트를 모두 지우고 보낸 것을 새로 넣는다.
    한 트랜잭션이라 반쪽 상태가 남지 않는다. 빈 목록이면 노트를 모두 지운다.
    """
    pattern, _project, member = _find(db, user, pattern_id)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)

    db.execute(delete(MidiNote).where(MidiNote.pattern_id == pattern.pattern_id))
    db.add_all(
        MidiNote(
            pattern_id=pattern.pattern_id,
            pitch=note.pitch,
            start_tick=note.start_tick,
            length_tick=note.length_tick,
            velocity=note.velocity,
            channel=note.channel,
            release_velocity=note.release_velocity,
            is_muted=note.is_muted,
            probability=note.probability,
            velocity_deviation=note.velocity_deviation,
        )
        for note in data.notes
    )
    db.commit()
    return to_response(pattern, _notes(db, pattern.pattern_id))
```

`delete_pattern` 은 클립이 쓰지 않는 패턴만 지웁니다.

```
def delete_pattern(db: Session, user: User, pattern_id: int) -> None:
    """패턴 삭제 (FR-2196). OWNER·EDITOR 만. 클립이 쓰고 있으면 400 으로 막는다.

    클립이 가리키는 패턴을 지우면 그 클립이 빈 껍데기가 되므로, 마지막 클립을 지운
    화면이 패턴 정리를 이어서 할 수 있게 남은 클립이 없을 때만 지운다.
    """
    pattern, _project, member = _find(db, user, pattern_id)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    if clip_service.pattern_is_used(db, pattern.pattern_id):
        raise AppError(ErrorCode.BAD_REQUEST, 400, "Pattern is used by clips.")
    db.delete(pattern)
    db.commit()
```

문법 풀이

식	뜻
`project_service.get_membership(db, project_id, user)`	멤버인지 확인하고, 남의 프로젝트면 있는지조차 알려 주지 않고 404로 막습니다
`project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	VIEWER가 저장이나 삭제를 시도하면 403으로 막습니다
`db.execute(delete(MidiNote).where(...))`	이 패턴의 노트를 모두 지웁니다. WHERE 이 없으면 표 전체가 지워집니다
`db.add_all(...)`	지운 자리에 보낸 노트를 한꺼번에 넣습니다
`for note in data.notes`	보낸 목록의 노트를 하나씩 꺼내 `MidiNote` 로 만듭니다
`_notes(db, pattern_id)`	교체가 끝난 뒤 DB 에서 노트를 새로 읽어 옛 목록과 섞이지 않게 합니다
`to_response(pattern, [])`	만들기 직후엔 노트가 없으므로 빈 목록으로 응답합니다
`clip_service.pattern_is_used(db, pattern_id)`	이 패턴을 가리키는 클립이 있으면 참이며, 그러면 400으로 막습니다
`NOTES_MAX = 5000`	한 번에 받는 노트 최대 개수입니다. 더 보내면 422가 납니다

3. 왜 이렇게

선택	이유
노트를 고치지 않고 통째로 교체	피아노롤은 지금 보이는 노트 전부를 다시 보냅니다. 삭제와 삽입을 한 트랜잭션으로 해서 반쪽 상태가 남지 않습니다
교체가 끝나면 `_notes` 로 다시 읽음	세션에 남은 옛 노트 목록과 섞이지 않도록 DB 에서 새로 읽습니다
빈 목록도 받음	노트를 모두 지운 빈 패턴을 저장할 수 있습니다
노트 최대 5000개로 제한	실수로 아주 큰 목록을 보내 DB 가 멈추는 것을 막습니다
패턴 삭제 전에 클립 사용 검사	클립이 가리키는 패턴을 지우면 그 클립이 빈 껍데기가 되므로, 쓰이는 패턴은 400으로 막고 쓰이지 않는 패턴만 지웁니다
패턴 목록 응답에 노트를 담지 않음	목록은 가볍게 두고, 노트는 상세 조회에서 받습니다 (FR-232, FR-233)
`note_id` 를 요청에서 받지 않음	통째 교체라 노트 번호를 서버가 새로 매깁니다

4. 검증

`tests/test_pattern_routes.py` 열두 건이 모두 통과합니다. `test_create_pattern_201` 은 만들기 201, `test_create_pattern_viewer_is_403` 는 VIEWER 거절을, `test_replace_notes` 는 노트 통째 교체 결과를 확인합니다. `test_replace_notes_validates_pitch` 는 음높이 범위 422, `test_viewer_cannot_save_notes` 는 노트 저장 403, `test_non_member_gets_404` 는 남의 패턴 404를 확인합니다. `test_delete_pattern_204` 는 쓰이지 않는 패턴 삭제, `test_delete_pattern_used_by_clip_is_400` 은 클립이 쓰는 패턴 삭제 거절을 확인합니다. 같은 규칙을 `tests/test_clip_model.py` 의 `test_pattern_used_by_clip_cannot_be_deleted` 가 DB 쪽에서도 봅니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다
화면 확인	피아노롤에서 노트를 찍고 저장한 뒤, 타임라인에서 패턴을 지우는 동작을 시퀀서에서 확인합니다